

JPA Caching

L1 Cache and L2 Cache

Chapter 4.6

L1 Cache	Persistence Context — automatic, per-transaction
L2 Cache	SessionFactory-scoped, shared, opt-in
Enabling L2	Spring Boot + Ehcache / Caffeine setup
Query Cache	Cache JPQL result sets, not just entities
Eviction	Staleness, TTL, and eviction strategies
L1 vs L2 Table	Side-by-side comparison
When to Use	Guidance on choosing L2 cache wisely

1 L1 Cache (Persistence Context)

Automatic, per-transaction, identity guarantee

The L1 cache is the first and most fundamental caching layer in JPA/Hibernate. It lives inside the Persistence Context (also called the Session in Hibernate). Every EntityManager has its own L1 cache that is active for the duration of one transaction. When you load an entity, Hibernate stores it in the L1 cache. If you load the same entity again within the same session, Hibernate returns the already-loaded object — no database query is made. This is completely automatic; you do not configure or enable anything.

The L1 cache also provides an identity guarantee: within one session, `em.find(User.class, 1L)` always returns the exact same Java object reference. When the session or transaction closes, the L1 cache is discarded.

Java Example

```
// L1 Cache demo – only ONE SQL SELECT is issued
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();

User u1 = em.find(User.class, 1L); // SQL: SELECT * FROM users WHERE id=1
User u2 = em.find(User.class, 1L); // No SQL – returned from L1 cache

System.out.println(u1 == u2);      // true – same object reference

em.getTransaction().commit();
em.close();                        // L1 cache destroyed here
```

Scope Diagram

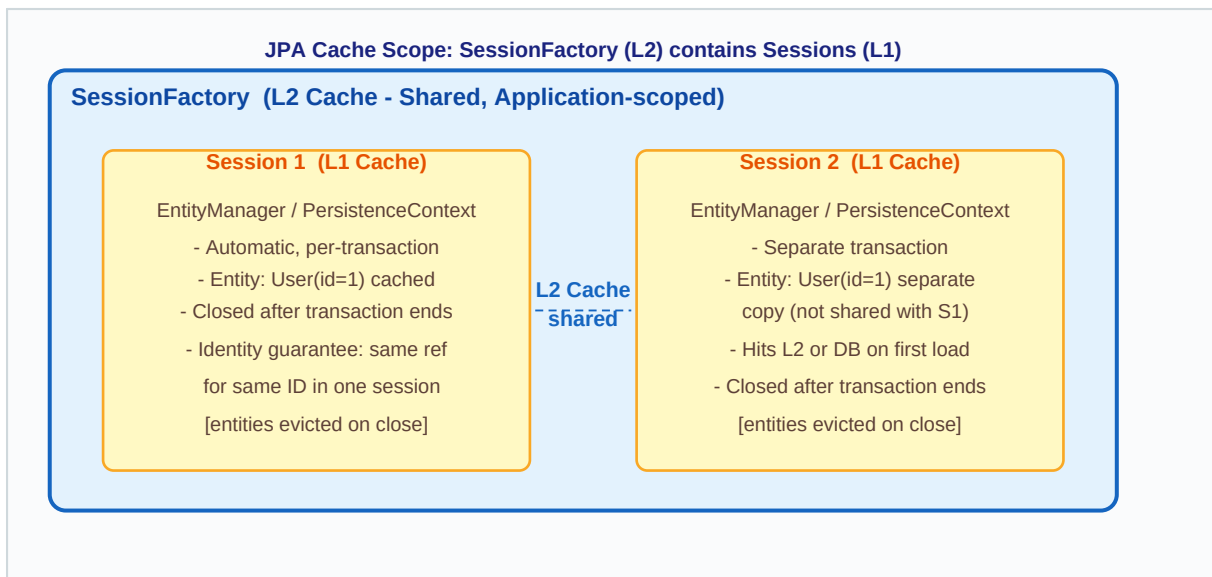


Figure 1 — Session (L1) scope nested inside SessionFactory (L2) scope

Key Rules

- L1 cache is always ON — you cannot disable it.
- Scope: one EntityManager (one transaction). Closed = cache gone.
- Identity guarantee: same ID in same session == same Java object.
- `em.clear()` wipes the L1 cache mid-transaction (use with caution).
- `em.detach(entity)` removes a single entity from L1 cache.

Q1. What is the L1 cache in JPA?

It is the Persistence Context (EntityManager/Session). It caches entities automatically within one transaction so that repeated finds for the same ID do not hit the database.

Q2. Can you turn off the L1 cache?

No. The L1 cache is mandatory in JPA. Every EntityManager always has an active L1 cache.

Q3. What happens to L1 cache when the transaction ends?

The EntityManager (and its L1 cache) is closed. All cached entities become detached objects with no backing cache.

Q4. What does the identity guarantee mean?

Within one session, calling `em.find(User.class, 1L)` twice returns the same Java object reference (`==`). Two different sessions would get two separate objects.

Q5. How do you manually clear the L1 cache?

Call `em.clear()` to evict all entities from the current Persistence Context, or `em.detach(entity)` to evict a single entity.

2 L2 Cache

SessionFactory-scoped, shared across sessions, opt-in

The L2 cache is an optional, application-wide cache managed by the SessionFactory (or EntityManagerFactory). Unlike L1, it is shared across all sessions and transactions. When Session A loads User(id=1) and Session B later requests the same user, Hibernate finds it in L2 and skips the database entirely. L2 cache is opt-in: you must enable it in configuration and annotate entities with `@Cacheable` (JPA) or `@Cache` (Hibernate).

Popular L2 cache providers include Ehcache, Caffeine (via cache2k or JCache), Infinispan, and Hazelcast. Hibernate acts as the coordinator — it stores, invalidates, and retrieves entries as needed.

Java Example

```
// Entity annotated for L2 caching
@Entity
@Cacheable // JPA: enable L2 for this entity
@org.hibernate.annotations.Cache(
    usage = CacheConcurrencyStrategy.READ_WRITE)
public class Product {
    @Id private Long id;
    private String name;
    private double price;
}

// Session A loads Product(id=5) -> SQL hit, stored in L2
// Session B loads Product(id=5) -> no SQL, served from L2 cache
```

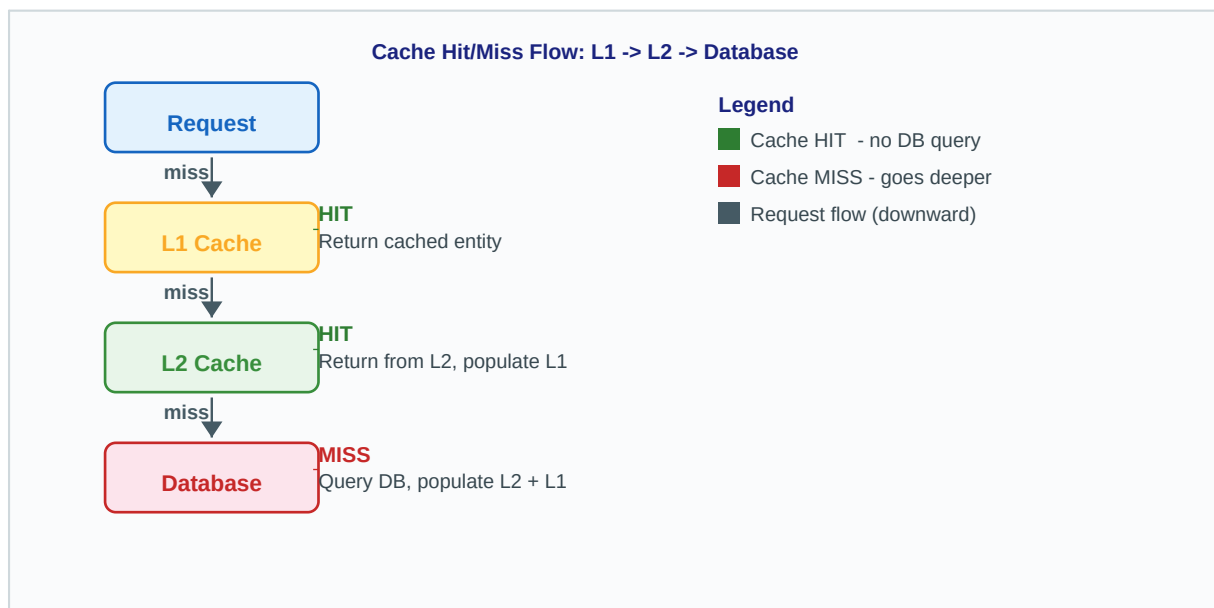


Figure 2 — Cache hit/miss flow: L1 check -> L2 check -> Database

Key Rules

- L2 cache is OPT-IN — enable in config and annotate entities.
- Scope: SessionFactory / EntityManagerFactory (whole application).
- L2 is checked only after L1 misses.
- Choose a CacheConcurrencyStrategy: READ_ONLY, READ_WRITE, NONSTRICT_READ_WRITE, TRANSACTIONAL.

- READ_ONLY is fastest and safest for truly immutable data.

Q1. What is the main difference between L1 and L2 cache?

L1 is per-session (destroyed when EntityManager closes). L2 is application-wide (shared across all sessions and survives transaction boundaries).

Q2. How do you mark an entity for L2 caching?

Add @Cacheable on the entity class and configure a cache provider. Optionally add @Cache to specify the CacheConcurrencyStrategy.

Q3. What is CacheConcurrencyStrategy?

It controls how Hibernate handles concurrent reads and writes to L2. READ_ONLY suits immutable data; READ_WRITE suits data that changes but must be consistent.

Q4. Does L2 cache help in all situations?

No. L2 helps most when the same entities are read frequently across many sessions. It hurts for frequently-updated data due to cache invalidation overhead.

Q5. Which cache providers work with Hibernate L2?

Ehcache, Caffeine (via JCache/cache2k), Infinispan, Hazelcast, Redis (via third-party adapters) are all popular choices.

3 Enabling L2 Cache in Spring Boot

Ehcache / Caffeine + @Cache annotation

Spring Boot makes it straightforward to enable L2 caching. You add a cache provider dependency (Ehcache or Caffeine), set two Hibernate properties in application.properties, and annotate your entities. That is all that is needed for a working L2 cache.

Step 1: Maven Dependency (Ehcache)

```
<!-- pom.xml -->
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-jcache</artifactId>
</dependency>
<dependency>
  <groupId>org.ehcache</groupId>
  <artifactId>ehcache</artifactId>
</dependency>
```

Step 2: application.properties

```
# Enable L2 cache
spring.jpa.properties.hibernate.cache.use_second_level_cache=true
spring.jpa.properties.hibernate.cache.region.factory_class=\
  org.hibernate.cache.jcache.JCacheRegionFactory
spring.jpa.properties.hibernate.javax.cache.provider=\
  org.ehcache.jsr107.EhcacheCachingProvider
```

Step 3: Annotate the Entity

```
@Entity
@Cacheable
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Country {
  @Id
  private Long id;
  private String name;
  private String code;
}
```

Key Rules

- Add hibernate-jcache + ehcache (or caffeine) to pom.xml.
- Set hibernate.cache.use_second_level_cache=true in properties.
- Configure the factory_class to match your cache provider.
- Annotate each entity with @Cacheable + @Cache(usage=...).
- Use spring.jpa.show-sql=true to confirm cache hits (no SQL logged).

Q1. What dependency is needed for Ehcache L2 in Spring Boot?

Add hibernate-jcache (Hibernate JCache bridge) and ehcache (the actual Ehcache library) to your pom.xml or build.gradle.

Q2. What is the minimum application.properties change to enable L2?

Set spring.jpa.properties.hibernate.cache.use_second_level_cache=true and provide the region.factory_class pointing to your provider.

Q3. Do all entities get L2 cached automatically after enabling it?

No. You must explicitly annotate each entity with `@Cacheable`. Hibernate will not cache entities without that annotation.

Q4. How does Caffeine work as an L2 provider?

Caffeine can be used via the JCache (JSR-107) API. Add `caffeine` + `cache2k-jcache` dependencies and point the `factory_class` to the Caffeine JCache provider.

Q5. How can I verify L2 cache is working?

Enable `spring.jpa.show-sql=true`. If the same entity is loaded twice across two sessions and only one SQL SELECT appears, the L2 cache is working.

4 Query Cache

Cache JPQL result sets, not just entities

The Query Cache stores the result sets of JPQL (or Criteria API) queries — specifically, the list of entity IDs returned by a query. It works together with L2 cache: the Query Cache holds the IDs, and the actual entity data is fetched from L2 (or the DB if not cached). This is useful when the same parameterized query is executed repeatedly and the underlying data rarely changes.

The Query Cache must be explicitly enabled in configuration AND each query must be marked as cacheable with `.setHint()`. It is best for stable, read-heavy queries like country lists, category trees, or config tables.

Java Example

```
// application.properties
// spring.jpa.properties.hibernate.cache.use_query_cache=true

// In your repository or service:
TypedQuery<Country> q = em.createQuery(
    "SELECT c FROM Country c WHERE c.region = :r", Country.class);
q.setParameter("r", "Europe");

// Mark this query result as cacheable
q.setHint("org.hibernate.cacheable", true);
q.setHint("org.hibernate.cacheRegion", "country.europe");

List<Country> results = q.getResultList();
// Second call with same params: served from Query Cache, no SQL
```

Key Rules

- Enable: `hibernate.cache.use_query_cache=true` (in addition to L2 config).
- Each query must opt in via hint `'org.hibernate.cacheable'=true`.
- Query Cache stores entity IDs only — actual data comes from L2.
- If an entity in the result is evicted from L2, Hibernate re-queries DB.
- Best for stable, frequently-run queries (e.g., lookup tables, config).

Q1. What does the Query Cache actually store?

It stores the list of primary key IDs returned by a JPQL query, not the full entity data. The actual entity objects are fetched from the L2 entity cache.

Q2. How do you enable the Query Cache?

Set `spring.jpa.properties.hibernate.cache.use_query_cache=true` in `application.properties`, and mark individual queries with the `cacheable` hint.

Q3. Can Query Cache work without L2 cache?

No. Query Cache depends on L2 cache. If L2 is disabled, enabling Query Cache alone has no effect.

Q4. What is a cache region in Query Cache?

A named partition in the cache store. Grouping related queries into a region allows targeted eviction (e.g., evict `'country.europe'` without clearing everything).

Q5. When is Query Cache a bad idea?

When the underlying data changes frequently. Every update to a cached entity invalidates the entire query cache region, causing constant cache misses.

5 Cache Eviction and Staleness

When L2 goes stale, eviction strategies

Stale cache data occurs when the database is updated but the L2 cache still holds the old value. This can happen when another application writes directly to the database bypassing Hibernate, or when bulk JPQL updates are used (they do not trigger entity lifecycle events). Hibernate handles single-entity updates automatically by evicting the cached entry on commit, but bulk operations and external DB writes bypass this mechanism.

Eviction strategies control how long data stays in L2 and how it is removed. Common strategies include TTL (Time To Live), TTI (Time To Idle), LRU (Least Recently Used) eviction, and manual programmatic eviction. Choosing the right strategy depends on how frequently the data changes.

Java Example — Manual Eviction

```
// After a bulk update or external DB write, evict stale entries manually:

// Evict one entity from L2:
emf.getCache().evict(Product.class, productId);

// Evict all cached Products:
emf.getCache().evict(Product.class);

// Evict everything from L2:
emf.getCache().evictAll();

// Hibernate-specific (more control):
sessionFactory.getCache().evictEntityData(Product.class, productId);
```

Ehcache TTL Configuration (ehcache.xml)

```
<cache alias="com.example.Product">
  <expiry>
    <ttl unit="minutes">10</ttl>    <!-- evict after 10 min -->
  </expiry>
  <heap>500</heap>                  <!-- max 500 entries -->
</cache>
```

Key Rules

- Single-entity saves/updates auto-evict L2 on transaction commit.
- Bulk JPQL updates (UPDATE ... WHERE) bypass entity lifecycle — evict manually.
- Use TTL to automatically expire entries after a set time.
- Use TTI to expire entries that have not been accessed for a period.
- External (non-Hibernate) DB writes cause staleness — plan your architecture.

Q1. What causes L2 cache staleness?

Bulk JPQL/SQL updates, direct JDBC writes, or writes from another application that bypass Hibernate entity lifecycle events. Hibernate cannot auto-evict what it does not know about.

Q2. Does Hibernate auto-evict L2 on entity save?

Yes, for single-entity save/merge/delete operations. Hibernate evicts the corresponding L2 entry when the transaction commits.

Q3. What is the difference between TTL and TTI?

TTL (Time To Live) evicts an entry after a fixed time since it was created. TTI (Time To Idle) evicts an entry after a fixed time since it was last accessed.

Q4. How do you evict all entities of a type from L2?

Call `emf.getCache().evict(MyEntity.class)` (JPA standard) or `sessionFactory.getCache().evictEntityData(MyEntity.class)` (Hibernate-specific).

Q5. What eviction policy is best for frequently-updated data?

For frequently-updated data, L2 cache is often inappropriate. If you must cache it, use a very short TTL (seconds) or `READ_WRITE` concurrency strategy with small max-entries.

6 L1 vs L2 Cache Comparison

Side-by-side feature table

The table below summarises the key differences between the L1 and L2 caches so you can quickly recall which cache applies in a given situation.

Feature	L1 Cache	L2 Cache
Scope	Per EntityManager (session)	Per SessionFactory (application)
Shared across sessions?	No — each session has its own	Yes — all sessions share one L2
Activation	Always on (mandatory)	Opt-in (config + annotation)
Entity identity	Guarantees same Java reference	Different object per session load
Lifetime	Until session/EM is closed	Configurable TTL / TTI
Eviction control	em.clear() / em.detach()	emf.getCache().evict() / TTL config
Provider needed?	None — built into Hibernate	Ehcache, Caffeine, Infinispan, etc.
Bulk update safe?	N/A (session-scoped)	No — must evict manually
Best for	All entity reads in a transaction	Read-heavy, rarely-changed entities

Q1. Which cache layer is checked first?

L1 cache is always checked first. Only when L1 misses (entity not in the current session) does Hibernate check L2. Only when L2 also misses does it query the database.

Q2. Can two sessions share L1 cache data?

No. Each EntityManager has its own isolated L1 cache. Two concurrent sessions always have separate L1 caches.

Q3. Does L2 cache require any coding effort?

Yes. You must add a cache provider dependency, set configuration properties, and annotate each entity you want cached. L1 requires zero coding effort.

Q4. Which cache is evicted when an entity is saved?

The L2 cache entry for that entity is evicted on transaction commit. The L1 cache still holds the updated managed entity until the session closes.

Q5. Can both L1 and L2 be active at the same time?

Yes — and in practice they always work together. L1 is always active. L2 is active when configured. Hibernate uses both in a layered fashion.

7

When to Use L2 Cache vs Avoid It

Guidance on choosing L2 cache wisely

L2 cache is a powerful tool, but it is not free. It adds memory overhead, increases configuration complexity, and introduces the risk of serving stale data. Used in the right places it can dramatically reduce database load; used incorrectly it creates subtle data consistency bugs.

USE L2 Cache when:

- [+] Data is read far more often than it is written (high read-to-write ratio).
- [+] Data is relatively stable — lookup tables, country lists, product categories, config values.
- [+] The same entities are accessed by many concurrent sessions.
- [+] Database round-trips are the performance bottleneck (confirmed by profiling).
- [+] Entities are immutable or near-immutable (use `READ_ONLY` strategy for maximum speed).
- [+] You can tolerate a small window of staleness (e.g., a few minutes TTL is acceptable).

AVOID L2 Cache when:

- [-] Data changes frequently (orders, inventory levels, user sessions, financial balances).
- [-] Strong consistency is required — stale reads are not tolerable.
- [-] Another application or JDBC writes directly to the same DB tables.
- [-] Bulk JPQL/SQL updates are heavily used — cache invalidation becomes unreliable.
- [-] The entity is already loaded only once per session (L1 is sufficient).
- [-] Memory is constrained and entity count is very large.

Decision Summary

Scenario	Recommendation
Country / language lookup table	L2 <code>READ_ONLY</code> — safe, fast
Product catalog (rare updates)	L2 <code>READ_WRITE</code> with TTL
User account (frequent updates)	Avoid L2 — use L1 only
Order / payment records	Never cache in L2
Application config key-value	L2 <code>READ_ONLY</code> or Query Cache
Shopping cart (session data)	Avoid JPA cache entirely

Q1. What kind of data is ideal for L2 caching?

Reference data that is read often and rarely or never changes: country codes, product categories, currency codes, system configuration. This data gives maximum benefit with minimum staleness risk.

Q2. Why should user account data not be in L2 cache?

User accounts are updated frequently (profile changes, password resets, role changes). L2 might serve outdated account data, causing security or UX issues.

Q3. What is the risk of L2 caching with bulk updates?

Bulk JPQL `UPDATE/DELETE` statements bypass Hibernate entity events. Hibernate cannot auto-evict the affected L2 entries. You must call `evict()` manually or the cache serves stale data.

Q4. Is L2 cache suitable for microservices sharing one database?

Be careful. If two services (with separate SessionFactories) share the same DB, each has its own L2 cache. Writes from Service A do not evict Service B's L2 cache, causing staleness.

Q5. What should you do before adding L2 cache?

Profile first. Identify the actual database bottleneck with tools like slow-query logs or APM. Add L2 cache only where evidence shows it will help. Do not pre-optimize.